

Filtrado Moving Average

Suavizado y descomposición de series temporales — ejemplos en R y Python

Dr. Marcelo Risk

Data Mining de Series Temporales · Maestría en Explotación de Datos y Descubrimiento de Conocimientos · FCEyN UBA

¿Qué es un filtro de media móvil (MA)?

Un promedio local que se desliza a lo largo de la serie

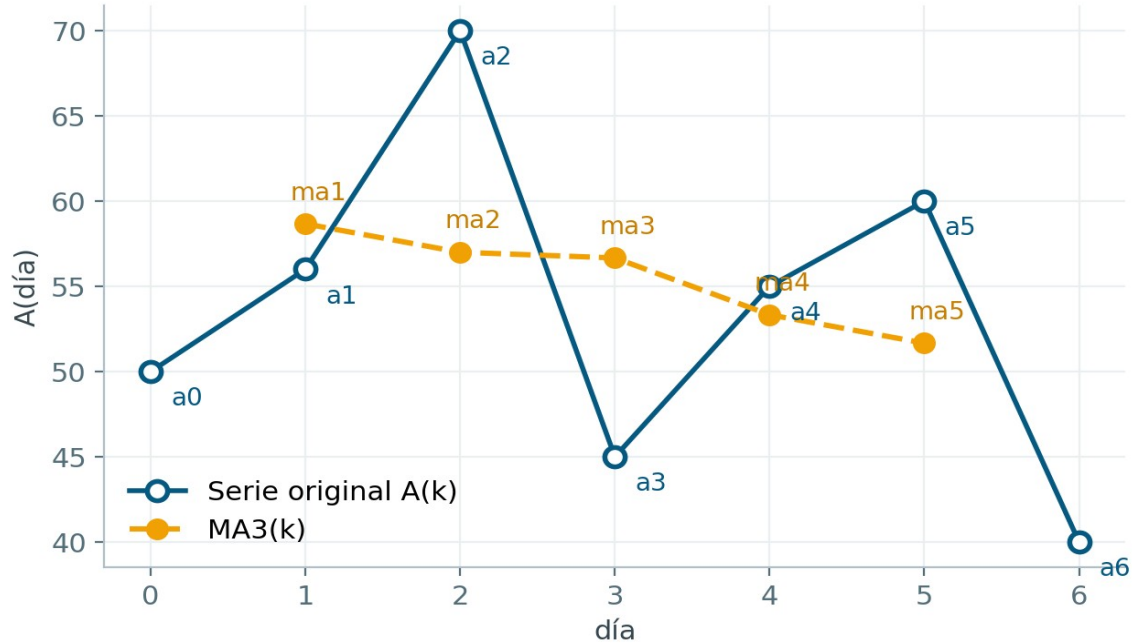
Para cada muestra k , la salida es el promedio de la muestra con sus vecinas. El filtro MA3 usa la muestra anterior, la actual y la posterior:

$$MA3(k) = [a(k-1) + a(k) + a(k+1)] / 3$$

- **Suaviza:** atenúa el ruido de alta frecuencia y deja pasar la tendencia lenta: es un pasa-bajos.
- **Simple:** solo requiere sumas y una división; es el filtro FIR más básico.
- **Bordes:** las primeras y últimas muestras no tienen todas sus vecinas: quedan sin definir o se resuelven de forma circular.

día	A(día)	MA3(día)
0	a0	—
1	a1	(a0+a1+a2)/3
2	a2	(a1+a2+a3)/3
3	a3	(a2+a3+a4)/3
4	a4	(a3+a4+a5)/3

Detalle del filtrado MA3



Lectura del gráfico

La serie original (azul) tiene picos abruptos como $a_2 = 70$ y $a_3 = 45$.

La salida MA3 (ámbar) los reparte entre vecinos: la curva filtrada es mucho más suave y sigue la forma general de la serie.

La familia de filtros MA

Más taps (coeficientes) = ventana más ancha = más suavizado

$$\text{MA5} \quad [a(k-2) + a(k-1) + a(k) + a(k+1) + a(k+2)] / 5$$

$$\text{MA7} \quad [a(k-3) + \dots + a(k) + \dots + a(k+3)] / 7$$

$$\text{MA}(n) \quad [\text{suma de } n \text{ muestras centradas en } k] / n$$

Trade-off clave: una ventana más ancha elimina más ruido, pero también atenúa y retrasa los cambios reales de la señal.

Señal de prueba: seno + ruido uniforme

El mismo experimento, lado a lado

R

```
N = 256
tiempo = 0:(N-1)
ciclos = 2
seno2 = sin(ciclos*2*pi*tiempo/N)

ruido = runif(N, min=-0.5,
              max=0.5)
senal = seno2 + ruido

plot(tiempo, senal, type='l')
hist(ruido)
```

Python

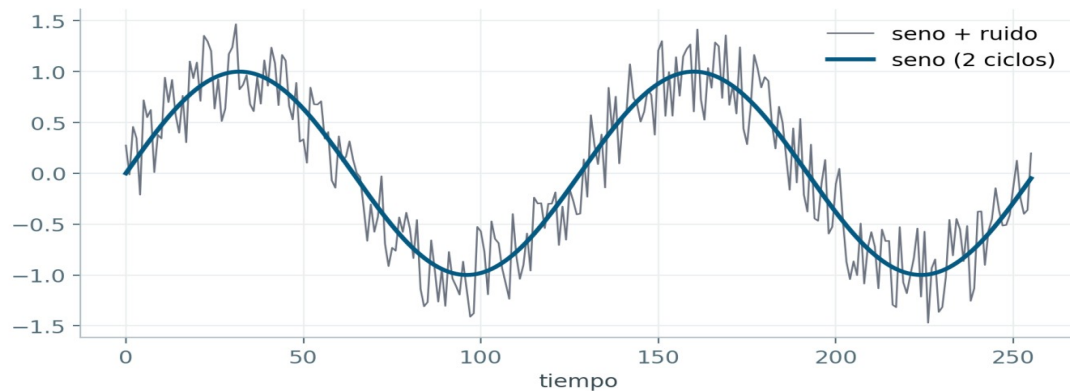
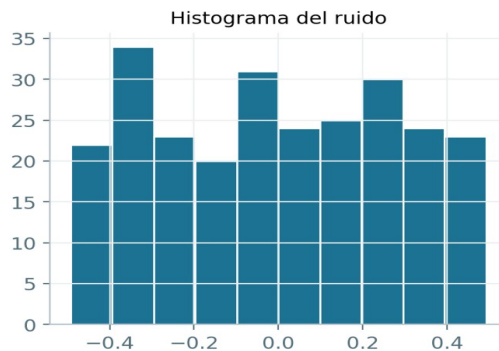
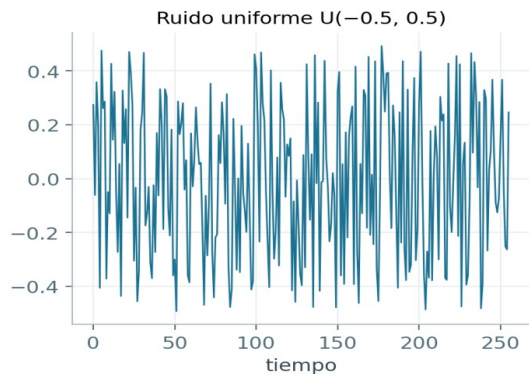
```
import numpy as np
import matplotlib.pyplot as plt

N = 256
t = np.arange(N)
ciclos = 2
seno2 = np.sin(ciclos*2*np.pi*t/N)

ruido = np.random.uniform(-0.5,
                           0.5, N)
senal = seno2 + ruido

plt.plot(t, senal)
plt.hist(ruido, bins=10)
```

La señal generada



Qué esperamos

El ruido uniforme tiene energía repartida en el rango de valores.

El seno de 2 ciclos concentra su energía en una sola armónica baja.

Un pasa-bajos debería recuperar el seno y descartar el resto.

Filtrado MA de 3 a 25 taps

filter() en R · convolución circular vía FFT en Python

R

```
ma3 = filter(senal,
  rep(1/3, 3), circular=TRUE)
ma7 = filter(senal,
  rep(1/7, 7), circular=TRUE)
ma25 = filter(senal,
  rep(1/25, 25), circular=TRUE)

lines(tiempo, ma3, col='red')
lines(tiempo, ma7, col='blue')
lines(tiempo, ma25,col='yellow')
```

Python

```
def ma(x, n):    # MA circular
    k = np.ones(n) / n
    K = np.fft.fft(k, len(x))
    X = np.fft.fft(x)
    return np.real(
        np.fft.ifft(X * K))

ma3, ma7, ma25 = (ma(senal, n)
  for n in (3, 7, 25))

# alternativa (sin bordes circ.):
import pandas as pd
ma7 = (pd.Series(senal)
  .rolling(7, center=True)
  .mean())
```

Tercera alternativa en Python: SciPy

Tres variantes del mismo MA7, con distinto tratamiento de fase y bordes

Python

```
from scipy import signal
from scipy.ndimage import uniform_filter1d

b = np.ones(7) / 7

# a) FIR causal: solo muestras pasadas
#     (introduce retardo de n//2 muestras)
ma7_causal = signal.lfilter(b, 1, senal)

# b) doble pasada ida y vuelta:
#     fase cero, sin retardo
ma7_fase0 = signal.filtfilt(b, 1, senal)

# c) centrado con bordes circulares =
#     filter(..., circular=TRUE) de R
ma7 = uniform_filter1d(senal, 7,
                       mode='wrap')
```

lfilter

causal: apto para tiempo real, pero desplaza la salida $n//2$ muestras.

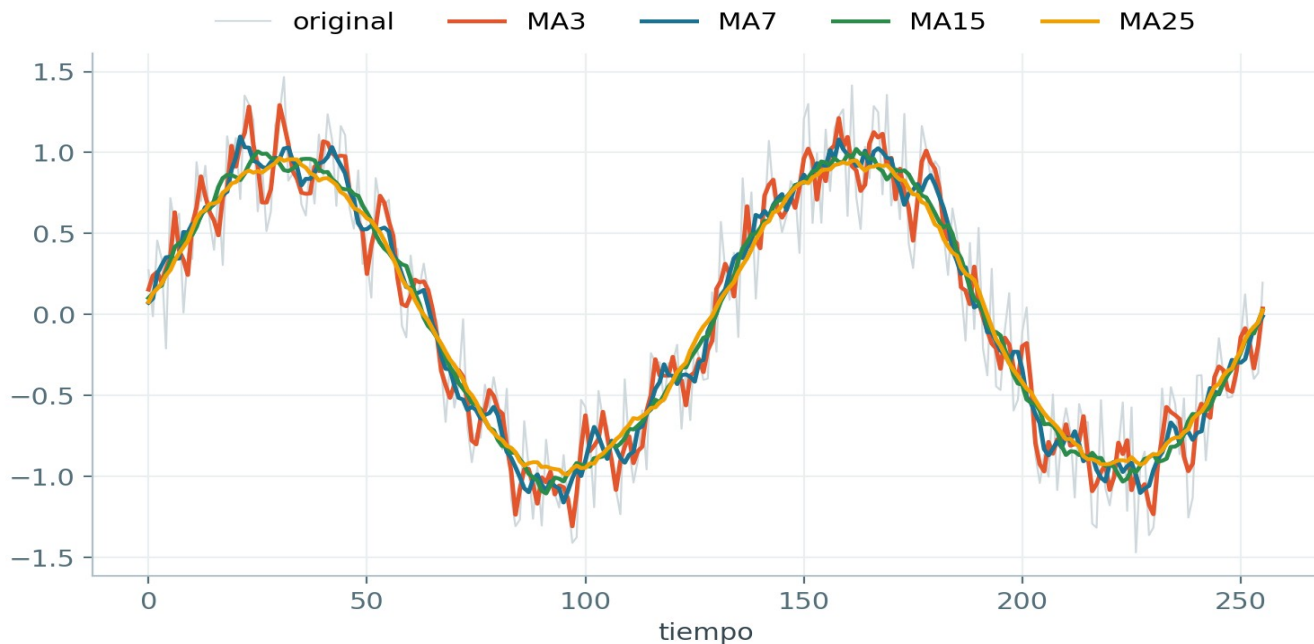
filtfilt

filtra dos veces (ida y vuelta): elimina el retardo, duplica el suavizado.

uniform_filter1d

ventana centrada con `mode='wrap'`: el equivalente exacto del ejemplo en R.

Resultado: más taps, más suavizado



MA3 aún deja pasar ruido; **MA25** casi reconstruye el seno puro, a costa de recortar levemente los picos.

Verificación en el dominio de la frecuencia

La FFT muestra qué armónicas sobreviven al filtro

R

```
fft.senal = fft(senal)
fft.ma3   = fft(ma3)
fft.ma25  = fft(ma25)

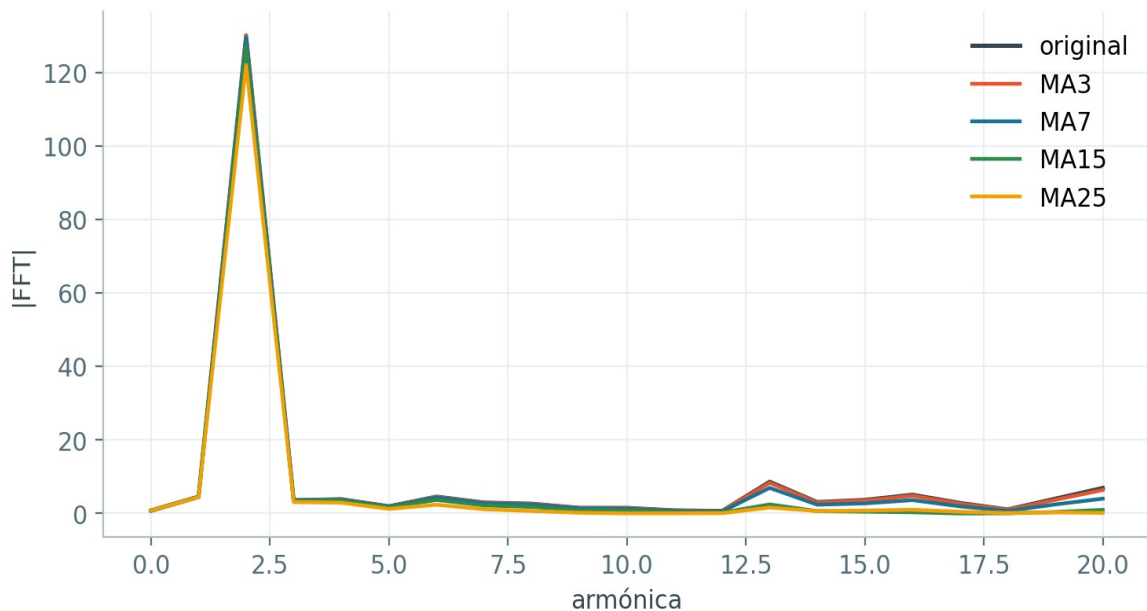
plot(tiempo, Mod(fft.senal),
      type='l', xlim=c(0,20))
lines(tiempo, Mod(fft.ma3),
      col='red')
lines(tiempo, Mod(fft.ma25),
      col='yellow')
```

Python

```
F      = np.fft.fft(senal)
F_ma3  = np.fft.fft(ma3)
F_ma25 = np.fft.fft(ma25)

plt.plot(np.abs(F)[:21],
         label='original')
plt.plot(np.abs(F_ma3)[:21],
         label='MA3')
plt.plot(np.abs(F_ma25)[:21],
         label='MA25')
plt.legend()
```

Espectro: zoom en las primeras armónicas

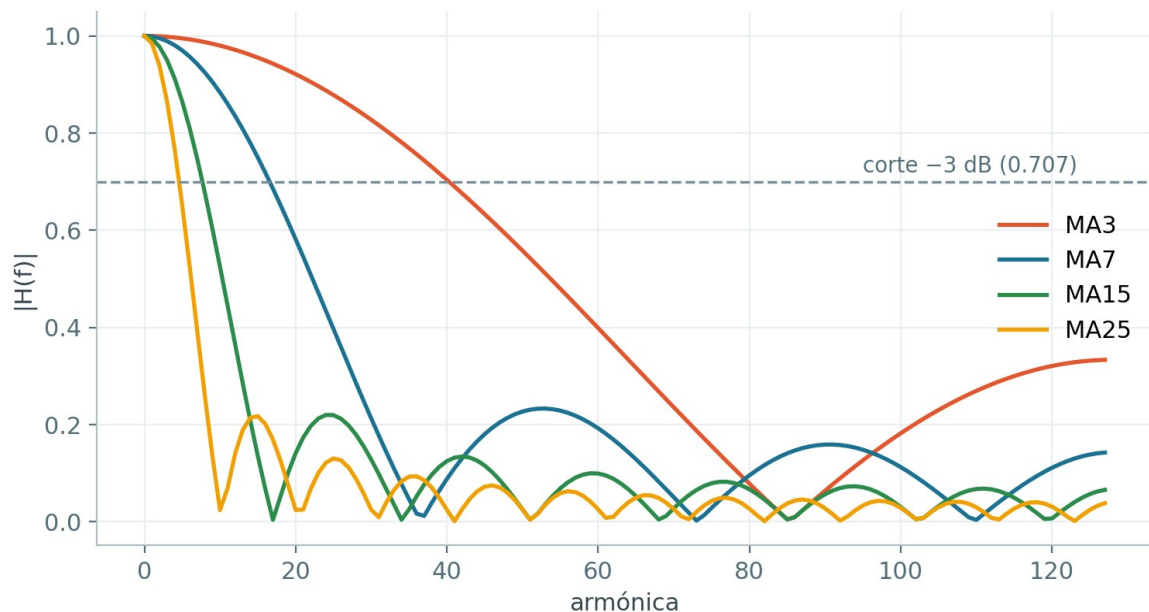


Observación

El pico en la armónica 2 (el seno) queda intacto en todos los filtros.

El piso de ruido baja cada vez más al aumentar los taps del MA.

Respuesta en frecuencia de los filtros MA



Python

```
h = np.zeros(N)
h[:3] = 1/3
H = np.abs(
    np.fft.fft(h))
plt.plot(H[:N//2])
plt.axhline(0.7,
    ls='--')
```

El MA es un pasa-bajos imperfecto: su frecuencia de corte baja al agregar taps, pero tiene lóbulos laterales que dejan pasar algo de alta frecuencia.

Descomposición de una serie temporal con MA

Serie diaria de un año: tendencia + componente mensual + residuo

1

Filtrar la serie original con un MA7

elimina la variación intra-semanal

2

Restar: original - MA7

residuo rápido (ruido diario)

3

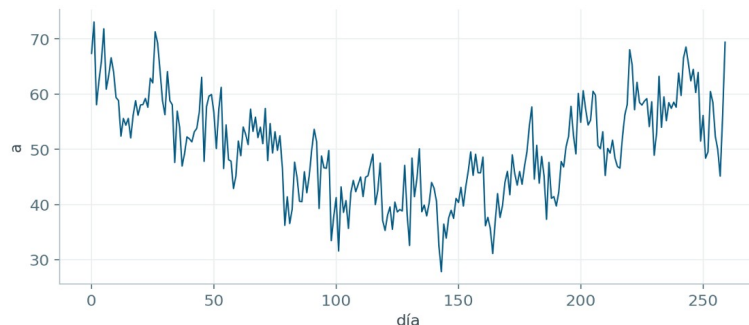
Filtrar la serie original con un MA21

queda solo la tendencia lenta

4

Restar: MA7 - MA21

aísla la componente mensual



La serie de ejemplo suma: **nivel constante + ciclo anual + ciclo mensual + ruido gaussiano.**

Descomposición: implementación

Cuatro líneas de filtrado y resta en cada lenguaje

R

```
dato = read.csv('TS_1.csv')

outMA7 = filter(dato$a,
  rep(1/7, 7), circular=TRUE)
difMA7 = dato$a - outMA7

outMA21 = filter(dato$a,
  rep(1/21, 21), circular=TRUE)
difMA21 = outMA7 - outMA21

plot(dato$t, outMA7, type='l')
lines(dato$t, difMA21,
  col='green')
```

Python

```
import pandas as pd

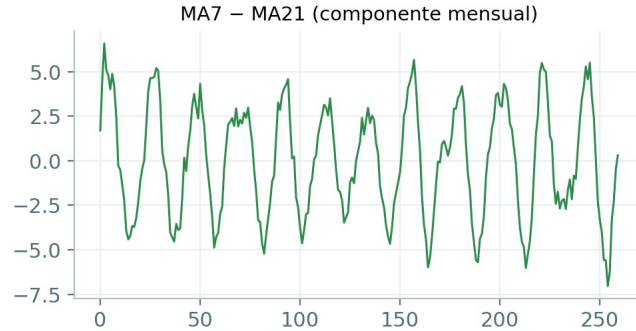
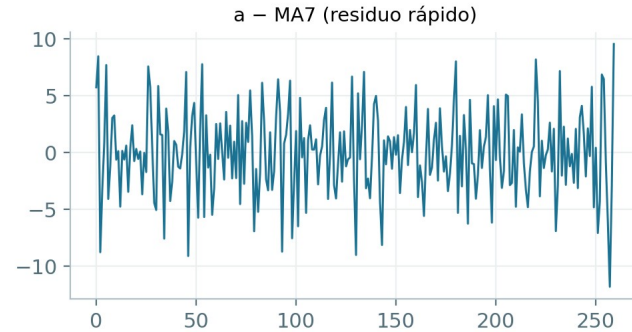
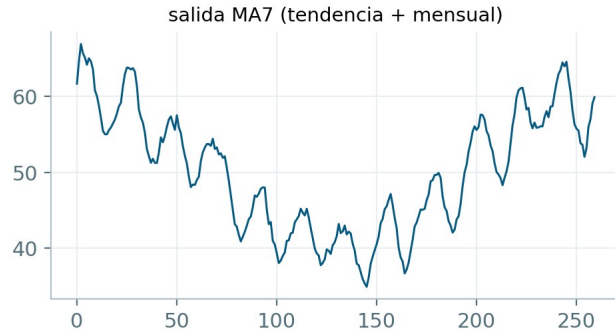
dato = pd.read_csv('TS_1.csv')
a = dato['a'].to_numpy()

outMA7 = ma(a, 7) # slide 7
difMA7 = a - outMA7

outMA21 = ma(a, 21)
difMA21 = outMA7 - outMA21

plt.plot(dato['t'], outMA7)
plt.plot(dato['t'], difMA21)
```

Cada componente en su escala de tiempo



MA7 conserva tendencia + ciclo mensual • MA21 deja solo la tendencia • la resta $MA7 - MA21$ aísla el ciclo mensual.

Ideas para llevarse

Pasa-bajos simple El MA promedia vecinos: atenúa ruido de alta frecuencia con costo computacional mínimo.

Taps = ancho de banda Más coeficientes $\rightarrow$ corte más bajo y más suavizado, pero mayor distorsión de picos y transiciones.

La FFT lo confirma Comparar espectros antes y después del filtro hace visible qué armónicas se conservan.

Descomposición por diferencias Combinando MA de distinto ancho (MA7, MA21) se separan tendencia, ciclo mensual y residuo.